

Compiladores

Aula 15: Conclusão da análise semântica

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Regra estática
- Equivalência, compatibilidade e conversão
- Vetores e funções
- Condições e comandos de laço

Regra estática

- A análise semântica interpreta a AST usando a tabela de símbolos e as regras da linguagem.



Nome

"x" refere-se a qual declaração?

Tipo

"int + bool" é uma operação válida?

Contexto

"break" ocorre dentro de um laço?

Saída: AST anotada e diagnósticos; ainda não há temporários, rótulos ou código de três endereços.

Regra estática

- A regra estática é decidida antes da execução; valores concretos podem exigir verificação em tempo de execução.

Compilação / semântica estática

```
int n;
n =true;
```

Tipos incompatíveis: rejeitar antes de executar.
Índice float em vetor: rejeitar antes de executar.

Execução / valores concretos

```
int i;
v[i];
```

Se i for int, o tipo do índice é válido.
Se seu valor estiver fora da faixa, detectar em execução — salvo prova estática.

TIPO INVÁLIDO ≠ VALOR FORA DOS LIMITES

Regra estática

- Cada uso deve apontar para uma declaração visível, com categoria e atributos preservados.

1. Buscar: escopo atual → pais
2. Classificar: variável • vetor • função
3. Associar: uso → símbolo resolvido
4. Diagnosticar: não declarado / categoria errada

POLÍTICA DE NOMES

- Duplicidade no mesmo escopo: erro
- Sombreamento interno: conforme MINIC
- Funções adiantadas/recursão: coletar assinaturas antes dos corpos, se permitido

Um identificador não resolvido recebe tipo sentinela ERROR: não gere erros derivados enganosos.

Regra estática

- Os tipos formam o vocabulário das regras.
- Para o núcleo MINIC:
  tipos primitivos, vetores e assinaturas de funções.

`int` **inteiro** aritmética, contagens e índices	`float` **real** valores fracionários	`bool` **lógico** condições e operadores lógicos
`void` **sem valor** retorno de função; não é valor de expressão	`array(n,T)` **vetor** tamanho + tipo do elemento	$(T_1,\ldots,T_n) \rightarrow R$ **função** parâmetros ordenados + retorno

Atribua um tipo a cada expressão;
não crie meios para “adivinhar” tipos de nomes não declarados.

Equivalência, compatibilidade e conversão

- Equivalência, compatibilidade e conversão são conceitos diferentes.
- A regra de conversão precisa ser única para atribuições, argumentos, inicializadores e retornos.

Equivalência

array(4,int) ≡ array(4,int)
≠ array(5,int)

Assinaturas: mesmos parâmetros, ordem e retorno.

Compatibilidade

int pode ser aceito onde se espera float.

Isso não torna int e float equivalentes.

Política MINIC

int → float: promoção permitida.

float → int: estreitamento implícito proibido.



A coerção é anotação semântica: a instrução que a executa fica para a etapa seguinte.

Equivalência, compatibilidade e conversão

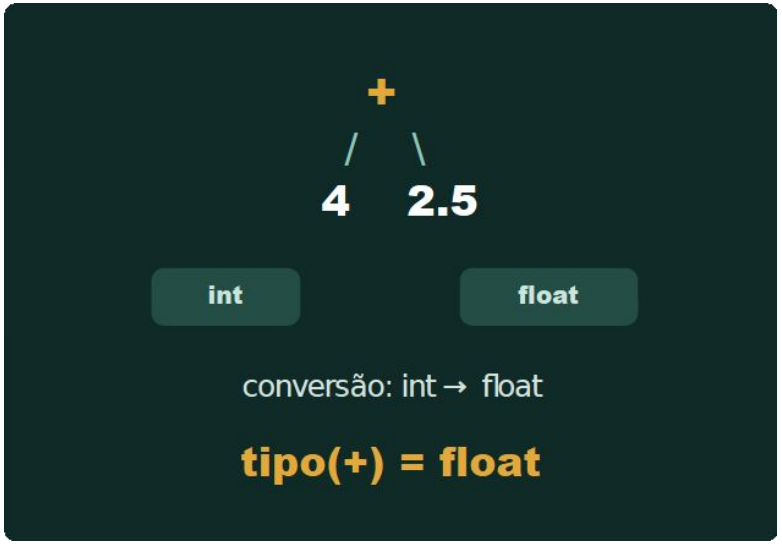
- A matriz de tipos é o contrato do verificador.
- Cada nó sintetiza um tipo a partir dos filhos, dos símbolos e da regra do operador.

Operação	Operando(s)	Resultado
Aritmética + − * /	int/float	int se ambos int; senão float
Módulo % (se existir)	int + int	int
Relacionais < ≤ > ≥	numéricos	bool
Igualdade == !=	comparáveis / compatíveis	bool
Lógicos && || !	bool	bool
Atribuição	destino + valor compatível	comando; sem valor

bool não é número: true * 2 é erro. Comparações produzem bool — não o tipo dos operandos.

Equivalência, compatibilidade e conversão

- A inferência de tipo é obtida pela AST, ao subir pelos nós da árvore.
- Exemplo misto: a promoção muda o tipo comum, não a regra da operação.



Operação válida

4 + 2 → int
4 + 2.5 → float (promoção)

Operação inválida

true * 2 → ERROR
"*" exige operandos numéricos.

Regra de ouro
Todo nó: tipo válido ou ERROR.

Equivalência, compatibilidade e conversão

- Valor-R fornece um dado; valor-L designa um destino.
- Além da compatibilidade de tipos, confirme se o lado esquerdo pode ser gravado.

Destino gravável

n

variável

Destino gravável

v[i]

elemento do vetor

Não gravável

4

literal

Não gravável

a + b

expressão

Não gravável

f()

chamada

Válido

```
float r; int n;
r = n;  // promoção permitida
```

Inválido

```
int n; bool r;
n = r;  // estreitamento proibido
```

Inicializadores, atribuições, argumentos e retornos consultam a mesma função de compatibilidade.

Vetores e funções

- Vetores acrescentam categoria, dimensão e tipo de elemento.
- O acesso v[i] é um nó semântico próprio: verifique objeto, índice, resultado e uso como destino.



float notas[4];

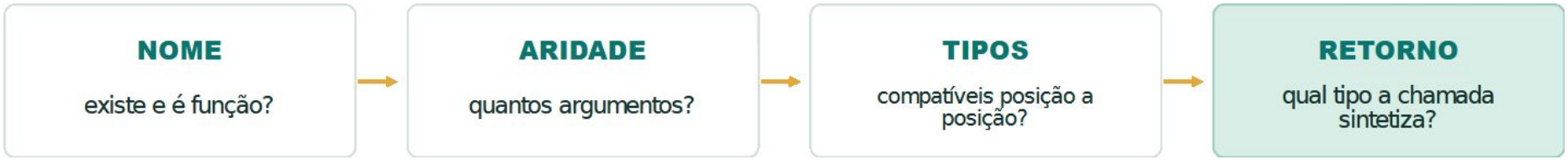
notas[i] = 8.5; ✓ notas[1.5] = 8; ×

Tipo ≠ faixa

Índice float: erro estático.
Índice int variável fora dos limites: normalmente execução.
Literal fora da faixa: compilador pode provar.

Vetores e funções

- A assinatura de uma função permite validar cada chamada.
- A ordem das verificações evita mensagens repetidas e associa cada argumento ao parâmetro certo.



float f(int a, float b);

f(3, 2.5) → válido; resultado float
f(true) → aridade +argumento inválido
f(3) → falta argumento

Retorno void

g(); pode ser comando.
x =g(); erro: não há valor para atribuir.

Protótipos e definições devem concordar; recursão depende da política de visibilidade de assinaturas.

Vetores e funções

- As verificações do tipo do return e queda são feitas pelo fim.
- São duas perguntas: cada retorno combina com a assinatura? Todo caminho de função não void retorna?

int f()

return 3; ✓
return 2.5; ×
return; ×

void g()

return; ✓
return 3; ×
chamada não produz valor

float h()

return 2; ✓
Promoção int → float.

if (c) return 1; else return 2;

AMBOS OS RAMOS RETORNAM

if (c) return 1; // sem else

PODE CAIR PELO FIM: ERRO

while não garante retorno: pode executar zero vezes. Use análise de fluxo conservadora.

Condições e comandos de laço

- Condições e comandos de laço dependem do contexto.
- Tipos e contexto de execução são verificações distintas, mas ambas são semânticas.

if / while

Condição deve ser bool.
if (x !=0) ✓
if (x) × na política MINIC desta apostila.

break / continue

Permitidos somente dentro de laço.
Não é erro de tipo: é erro de contexto.

loopDepth

Ao entrar em while: +1.
Ao sair: –1.
Depth =0 → break/continue inválidos.

```
while (ok && i < n) {  if (i == 3) continue;  if (i > 8) break; }
```

Operadores lógicos exigem bool; comparações produzem bool.
Curto-circuito é regra de execução, não dispensa checagem estática.

Condições e comandos de laço

- A análise deve ser organizada em passagens com responsabilidades claras.
- Coletar assinaturas antes dos corpos é uma estratégia robusta quando a linguagem permite chamadas adiantadas.

- Coletar declarações globais / assinaturas
- Abrir escopos e inserir parâmetros e locais
- Resolver nomes; sintetizar tipos; validar comandos
- Analisar fluxo e consolidar diagnósticos

ANOTAÇÕES NA AST

- símbolo
- tipo
- coerção
- assinatura
- posição-fonte

Tipos de expressões: sintetizados de baixo para cima.
Tipo de retorno e loopDepth: contexto herdado.

Condições e comandos de laço

- Tratamento de erros: uma mensagem de erro bem definida aponta causa, contexto e expectativa.
- A mensagem deve ajudar a corrigir; a categoria estável ajuda a testar.

SEM001	NOME	x não declarado neste escopo
SEM003	TIPO	não é possível atribuir float a vetor
SEM005	CONDIÇÃO	esperado bool; recebido int
SEM007	CHAMADA	f espera 2 argumentos; recebeu 1
SEM010	CONTEXTO	break só pode ocorrer dentro de laço

ERRO primário → tipo sentinela ERROR → suprimir cascata,
continuar em regiões independentes

Condições e comandos de laço

- Casos de testes: separe falhas do parser, da resolução de nomes e do verificador semântico.

Nomes
declarado • ausente • duplicado • sombreado

Expressões
aritmética • bool • comparação • mistura

Vetores
índice int/float • escalar indexado • elemento

Funções
aridade • tipos • void • assinatura

Retornos
tipo • ausência • todos os caminhos

Controle
condição bool • break/continue dentro/fora

Inclua posição linha/coluna, erros independentes acumulados e teste explícito de supressão de cascata.

Condições e comandos de laço

- No exemplo, acompanhe nomes, tipos, promoção, condição e contexto do laço.

```
int soma(int a, int b) { return a + b; }
float promove(int n) { return n; }
int principal() {
  int i; float x; bool ok;
  i = soma(2, 3);
  x = promove(i) + 0.5;
  ok = x > 2.0;
  while (ok && i < 10) {
    i = i + 1;
    if (i == 7) continue;
    if (i > 8) break;
  }
  return i;
}
```

01 **soma(2,3)**
assinatura / aridade / tipo int

02 **promove(i) + 0.5**
int → float; resultado float

03 **x > 2.0**
comparação → bool

04 **break / continue**
válidos dentro do while

05 **return i**
compatível com int principal

Condições e comandos de laço

- A análise semântica está completa quando:
  - todos os nomes e categorias foram validados;
  - cada expressão tem tipo ou ERROR, conversões estão anotadas;
  - atribuições, vetores e chamadas respeitam suas regras;
  - retornos e comandos de controle respeitam o contexto;
  - diagnósticos são úteis;
  - testes positivos e negativos cobrem as regras.